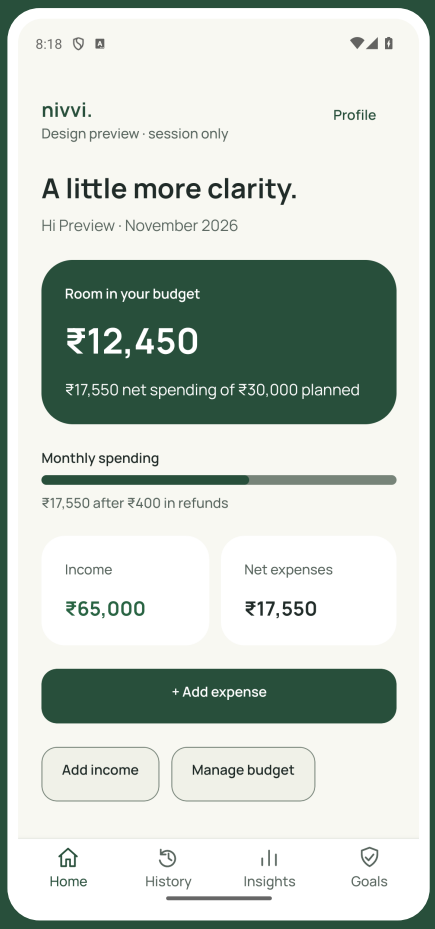
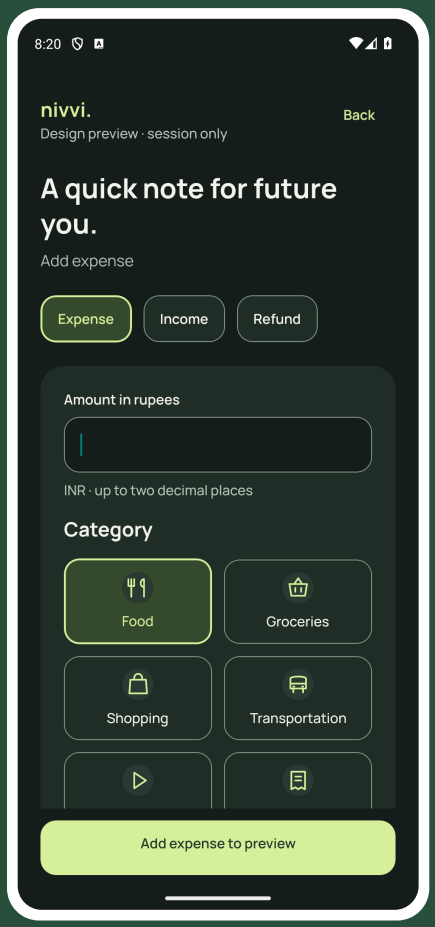
nivvi.

Your app.
Every layer, explained.

An illustrated beginner's guide, from the first tap on your phone to the tools that build the app.





REAL APP SCREENS + SIMPLE DIAGRAMS + PLAIN LANGUAGE

Prepared for Karthik I Current preview and future plans are labelled separately.

10 November 2026 / No prior programming knowledge needed

What you have today

You are looking at an interactive development preview, not a finished finance service.

IN USE TODAY

What already works

Home, expense/income/refund forms, History, categories, budgets, savings goals, Insights, profile and settings share sample data. Changing an entry updates the other screens immediately.

Where the data lives

Today's entries are held in the phone app's temporary memory. Closing/restarting the app or signing out can reset the preview. There is no SQLite ledger or server database connected yet.

What login means today

The email/password screens demonstrate the experience. They do not create a real account, authenticate with a server or send a reset email. Use sample details only.

Why your laptop is involved

This is a development build. Metro on the laptop supplies the app's JavaScript program to the phone. Metro is not a financial backend and does not store your expenses.

Important: UPI is currently a payment-method label. The app does not connect to a bank, read SMS, scan notifications or transfer money. Nim is original static artwork; original animated Rive files are still missing.

Your reading map

Read in order, or jump directly to the question you have.

REFERENCE

I want to understand the screens | Pages 4-8

Onboarding, Home, expenses, History, Insights, goals, themes and accessibility.

I want to understand the technology | Pages 9-23

Code, React Native, Expo, Metro, Android Studio, builds, state, design, motion, testing and Git.

I want to know what comes later | Pages 24-28

Backend, real login, databases, offline sync, Android widgets and approval gates.

I want exact package names | Pages 29-30

The actual installed direct mobile dependencies and what each one contributes.

I just want to open the app | Page 31

A practical daily checklist, sample preview details and launcher instructions.

I need a word explained | Pages 32-34

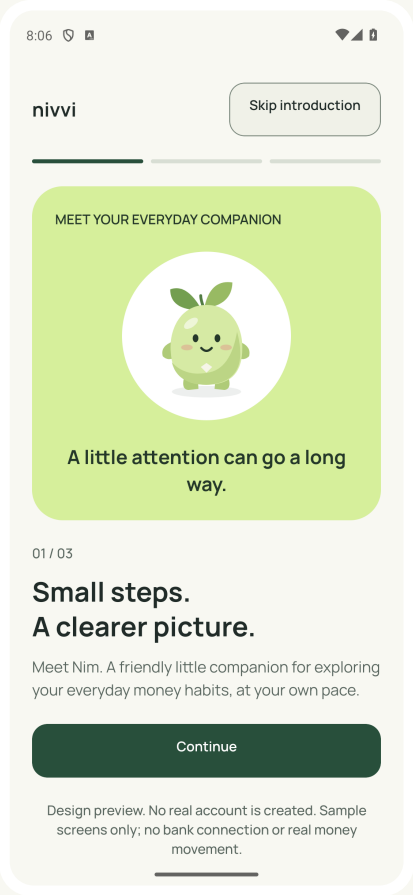
A quick glossary plus official documentation and local project evidence.

How pictures are used: product screenshots are real Phase 3 Android emulator captures. Architecture drawings are explanatory illustrations, not screenshots of implemented servers or widgets. Screenshots stay sharp when you zoom.

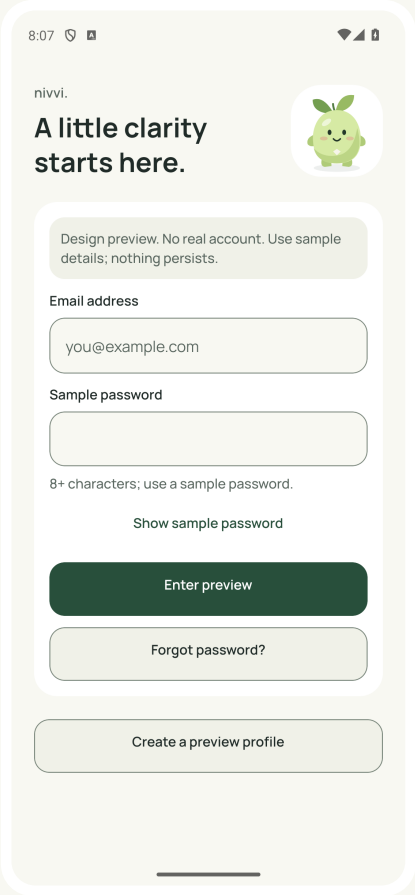
Getting past the launcher

The development launcher comes before the Nivvi app itself.

IN USE TODAY



1. Nivvi introduction



2. Preview sign-in

If you see 'Development servers': you are in Expo's launcher. It is waiting for the laptop's Metro server. Connect to that server first; then these Nivvi screens appear.

Inside Nivvi: finish the introduction or tap Skip introduction. Enter sample email **preview@example.com** and sample password **Preview123**, then tap **Enter preview**. This opens Home.

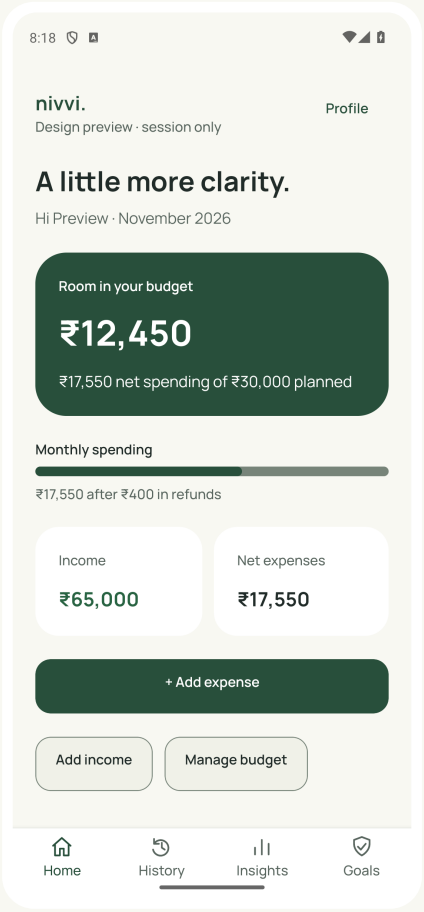
These are demonstration details

They are not a real username/password pair. There is no account registration service behind this preview. The profile and records reset with the session.

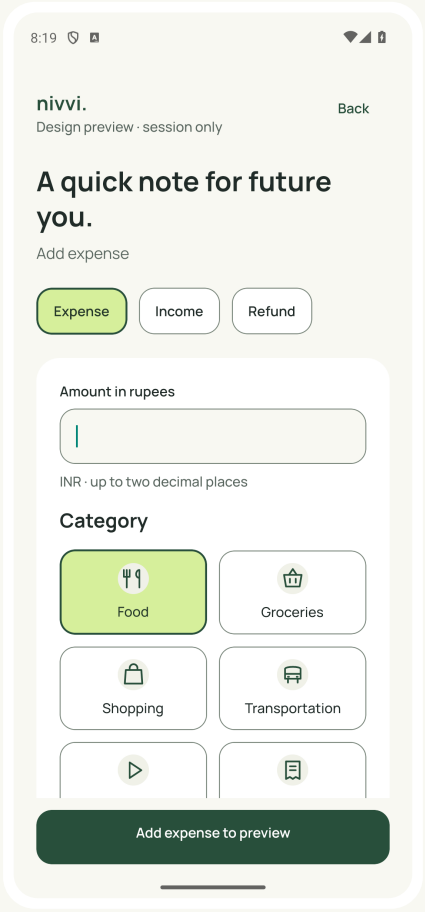
Home and Add Expense

A monthly overview and a form that changes it.

IN USE TODAY



Home: sample monthly totals



Add Expense: manual entry

Home: the sample monthly budget is ₹30,000. Expenses minus refunds are ₹17,550, so the remaining budget is ₹12,450. This is a spending-plan calculation, not a bank balance.

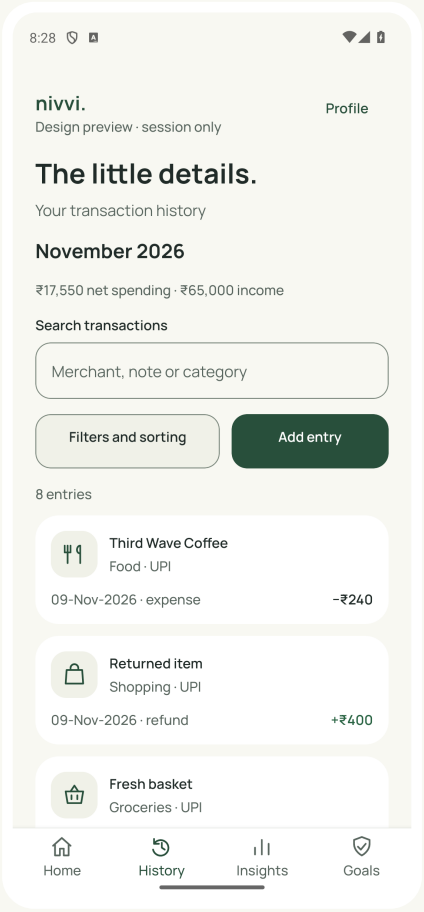
Add Expense: enter an amount, pick a category and save. Payment method defaults to UPI as metadata. Expand details to change date/payment method or add merchant and note. The Save action stays below the scrollable form.

Add Income records money received. A **refund** links to an existing expense and reduces net spending on its refund date. None of these actions performs a payment.

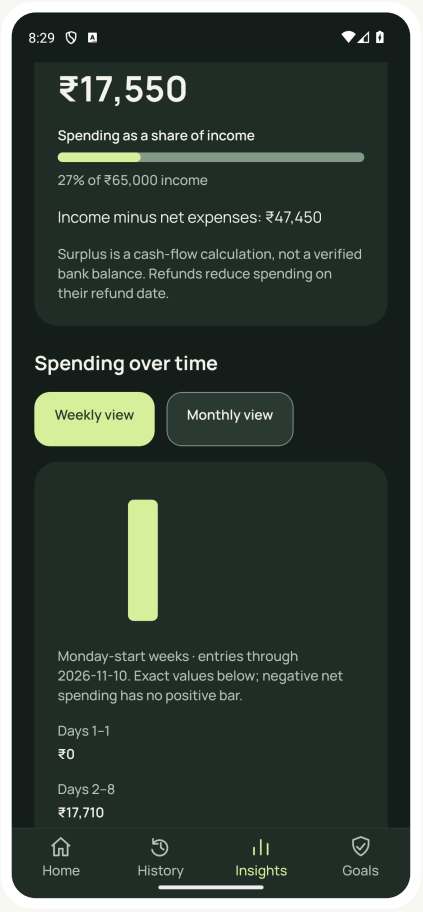
History and Insights

Your entries become a searchable history and a financial summary.

IN USE TODAY



History: real preview rows



Insights: derived weekly chart

History: search by merchant, note or category. The expandable panel contains month, type, category, payment-method and sorting controls. Tap a row to see details, edit it or confirm deletion.

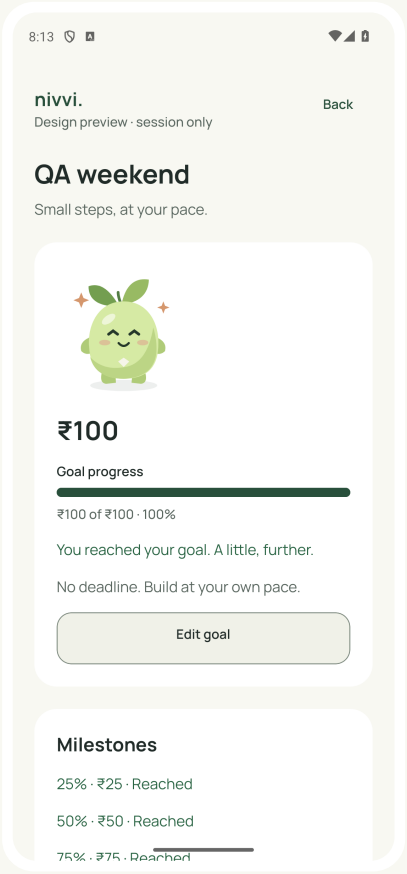
Insights: computes weekly/monthly spending, category and merchant summaries, and comparisons from those same records. It does not download bank statements or use an AI prediction model.

The sample data is concentrated early in the month, so a weekly chart can contain one large bar and several zero values. Exact values are also written as text; a negative net amount is retained in the numbers.

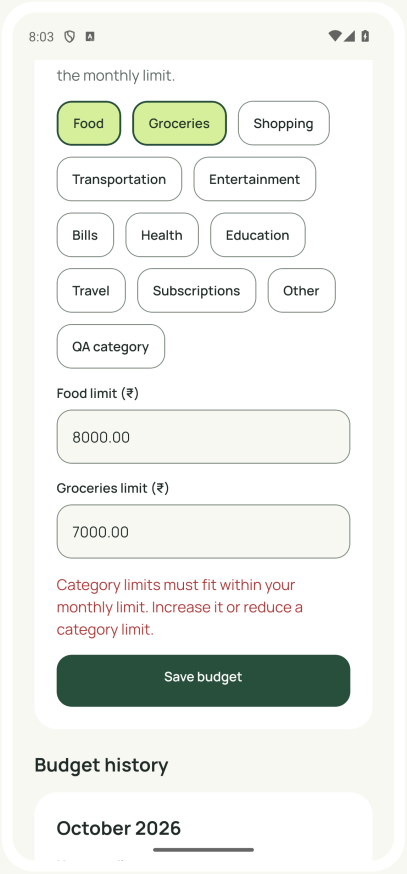
Budgets, goals and Nim

Planning tools give the numbers a purpose.

IN USE TODAY



A goal reaches its target



A budget rejects invalid allocation

Budgets: set a monthly limit and optional category limits. Combined category limits must fit inside the monthly limit. Errors explain what to change rather than silently saving an inconsistent plan.

Goals: set a target and track contributions. Reaching the target reveals milestones and Nim's happy artwork. A contribution records your intention; it does not transfer money or add a ledger expense.

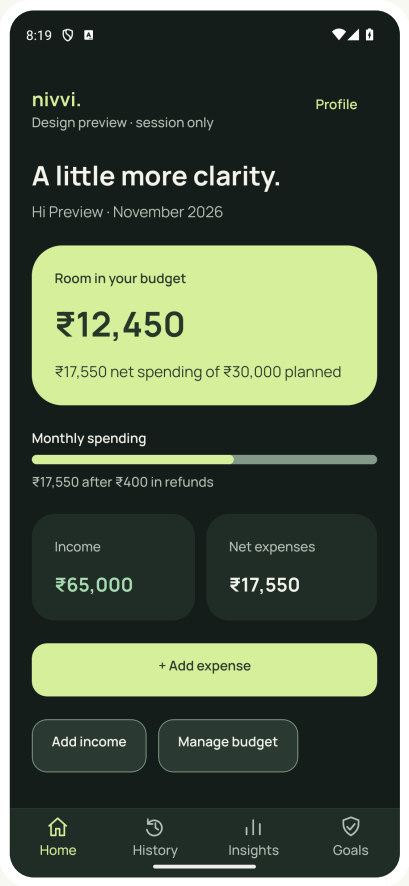
Who is Nim?

Nim is Nivvi's original companion. Different static states communicate calm, attention and celebration. Real Rive state-machine animation remains pending until original .riv assets are supplied and tested.

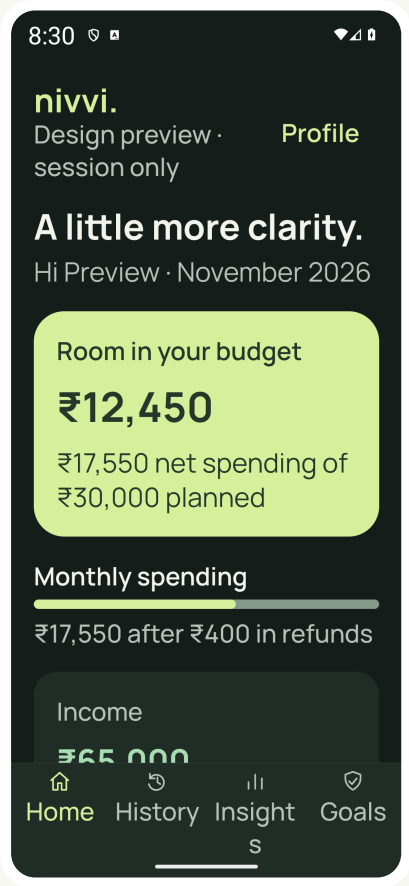
Light, dark and readable text

The same product has two deliberate themes and supports larger system text.

IN USE TODAY



Dark theme at normal text size



Actual 200% Android text setting

Theme: Settings can follow the system or use light/dark explicitly. Shared colour roles change together; dark mode uses its own surfaces and accents.

Readability: supporting text starts at 14sp and body text at 16sp. Android can enlarge it. Cards and navigation reflow, and the tab bar grows instead of cutting off tall labels; long labels may wrap.

Accessibility: controls have labels and meaningful states, and decorative motion can be reduced. Contrast checks passed, but a complete TalkBack speech/focus review and a full accessibility audit are still outstanding.

The complete picture today

Two working places today: the development computer and the phone.

IN USE TODAY

Your Windows laptop

Source files
Node.js + npm
Expo CLI + Metro
Android Studio + Android SDK



USB / ADB

Your Android phone

Installed development APK
Hermes + React Native
Nivvi screens
Temporary Zustand records

During development

The laptop builds/serves the program. The phone runs the interface and keeps preview entries in its own memory. A USB reverse connection lets the phone reach Metro at port 8081.

Future: API + PostgreSQL

A real backend will serve accounts and cross-device financial records. It is planned, not running behind today's app.

Three different jobs: Metro supplies developer code; a future API will process financial requests; a database will persist records. Calling all three 'the server' hides an important difference.

A normal release build bundles its application code and does not need your laptop or Metro. That future packaging is separate from adding a backend and separate from offline storage.

Code, files and folders

A project is a set of files that describe appearance, behaviour and build instructions.

IN USE TODAY

Source code - human-readable instructions written by developers.

Program / application - the running result of those instructions.

Component - a reusable interface piece, such as a button, card or input.

Library / package - reusable code provided as a dependency.

Terminal / PowerShell - a place to type commands that start tools or builds.

```
expense-tracker/
  apps/mobile/          # current Expo application
    app/                # navigable routes
    src/features/       # product screens + rules
    src/components/     # shared buttons/cards/etc.
    plugins/            # repeatable native settings
  design/               # brand tokens + artwork
  docs/                 # proposals + review evidence
  package.json          # workspace commands
  package-lock.json     # exact dependency tree
```

Monorepo means related projects live in one repository. Today the active npm workspace is mobile. Backend/shared/native-widget areas are planned as the corresponding phases arrive; a proposed folder is not proof of an implemented service.

JavaScript to native screens

Four related names, each with a different job.

IN USE TODAY

Technology	Plain meaning	Example in Nivvi
JavaScript	A programming language that describes behaviour.	Calculate a total or respond to a tap.
TypeScript	JavaScript with types checked by development tools.	Require an amount to be a number and a transaction type to be valid.
React	A component model for updating UI from changing state.	A total changes when the transaction records change.
React Native	Uses React to build a native mobile interface.	Android text, input, scrolling and native screen behaviour.

```
// Simplified teaching example, not the full source:
type Expense = { amountPaise: number };
const expense: Expense = { amountPaise: 12550 };
// A React Native component can display this amount.
```

TypeScript is not a database or a security system. It catches many coding mistakes before the app runs, but user input still needs runtime validation. A future server must validate and authorize requests too.

Most application code is .ts or .tsx. TSX allows component markup inside TypeScript. Native libraries also contain Kotlin, Java and C++ code; you do not manually write all of those layers for every screen.

What Expo adds

Expo supplies a coordinated development toolkit around React Native.

IN USE TODAY

Expo SDK

A versioned collection of compatible libraries, native modules and tools. This project uses Expo SDK 58 with React Native 0.88.1 and React 19.3.0. These versions are a selected set, not interchangeable numbers.

Expo CLI and config

The CLI is the command-line tool used by npx expo. app.config.ts describes the app name, package ID, icon, scheme, plugins and build environment. 'npx expo prebuild' generates the native Android project from that configuration.

Config plugins

Repeatable changes to generated native files. Our plugins preserve font-scale handling and select newer CMake on Windows. They avoid relying on edits that disappear when Android is regenerated.

Build/app	What it means here
Expo Go	A general-purpose preview app with a fixed set of native modules; not this project's native acceptance environment.
Development build	Our custom native app including Rive/Nitro and the Expo developer launcher. Uses Metro while developing.
Release build	Future distribution packaging with bundled code. Today's production preview routes are deliberately guarded; switching to release is not a shortcut to a finished app.

Node.js, npm and packages

Node.js runs tools; npm manages their packages.

IN USE TODAY

Node.js

A runtime that executes JavaScript outside a web browser. On your laptop it runs Expo CLI, Metro and other tooling. Node.js is also planned for the future backend, but no finance backend is implemented today.

npm and npx

npm installs dependencies and runs project scripts. npx launches a package's command, such as expo. A dependency is reusable code that the project needs; a development dependency mainly supports building or checking it.

package.json and package-lock.json

The first lists packages and commands. The lockfile records exact resolved versions so a fresh installation can reproduce the dependency tree. A ~ or ^ in a package entry describes a permitted update range; it is not the resolved installed version.

npm workspaces

Connect the mobile project to the root package setup. One root lockfile prevents separate installations from drifting apart. Version overrides keep React and React DOM on the selected compatible pair.

Examples: **npm test** runs automated tests; **npm run lint** checks code rules. These do not launch the user interface. A normal daily review does not require reinstalling all dependencies.

Metro, USB and localhost

Why '127.0.0.1:8081' appears when you run the app.

IN USE TODAY

Laptop: Metro

Bundles application code
Listens on port 8081
Runs under Node.js



Phone: dev client

Requests the program
Executes it with Hermes
Displays Nivvi

Localhost = this device

127.0.0.1 normally points back to the device making the request. On the phone it normally means the phone, not your laptop. 'adb reverse tcp:8081 tcp:8081' creates the USB mapping needed for this development connection.

Port = a numbered service entrance

8081 is the local port used by Metro here. It is not an account number, payment port or remote finance service. The HTTP URL tells the client where to fetch development code.

Metro and Fast Refresh

Metro resolves imports and packages the code. Fast Refresh updates many screen-code changes during development. Native dependency/configuration changes can still require rebuilding the Android app.

If Metro stops or USB reverse is lost, the launcher or a loading message can appear. The current app does not become a standalone offline app just because an APK is installed.

Android Studio and the SDK

An editor, a toolkit, a virtual phone and a physical phone are different things.

IN USE TODAY

Name	Job
Android Studio	The IDE: a graphical workspace for importing the Android project, syncing Gradle, building, running and debugging.
Android SDK	Software Development Kit: Android platform APIs, build tools and platform tools installed on the computer.
ADB	Android Debug Bridge: communicates with a connected device, installs APKs, reads logs and creates USB port mappings.
Emulator / AVD	Android Virtual Device: a phone simulated on the laptop. Pixel_7 API 34 was used for the main screen captures.
Physical device	Your actual phone. The I2220 phone was connected and showed Home/Add Expense after the server was opened.

Android version is not CPU architecture

An **API level** describes Android platform capabilities. An **ABI** describes the compiled machine-code interface for a processor family. Your phone build used **arm64-v8a**; the tested desktop emulator uses **x86_64**.

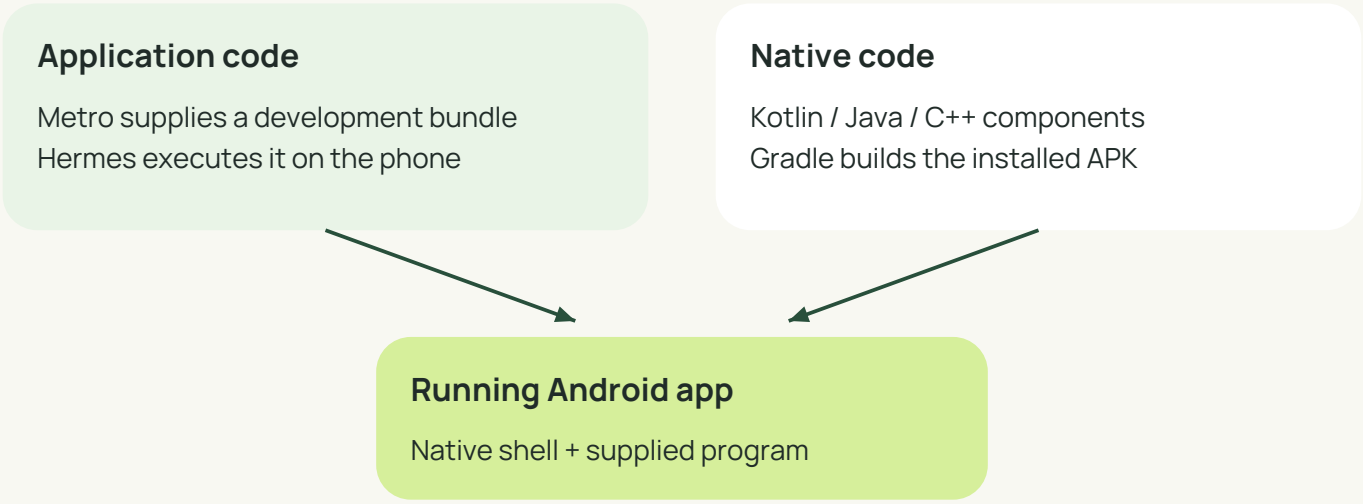
compileSdk chooses the Android APIs available while compiling. **targetSdk** selects the platform behaviour level the app targets. **minSdk** is the oldest supported Android API. They are not Expo or AGP version numbers.

An APK needs compatible native libraries for its target device. A build verified for ARM64 alone is not proof of a new x86_64 emulator build.

How Android builds the app

Two kinds of code meet inside the Android application.

IN USE TODAY



Tool / language	Its part of the build
Gradle + wrapper	Coordinates tasks; gradlew.bat uses the project's Gradle version.
AGP	Android Gradle Plugin: adds Android-specific build, packaging and native tasks.
JDK / JBR	Java Development Kit tooling runs Gradle. Studio bundles JetBrains Runtime (JBR).
Kotlin + Java	Languages used by the generated Android shell and native libraries; widget-specific Kotlin is still planned.
NDK + C++ + Clang	The native toolkit/language/compiler used by React Native and native modules.
CMake + Ninja	CMake configures the C++ build; Ninja executes its compile/link steps.
Hermes	The JavaScript engine running application logic on the phone. It is not Metro, Node.js or a database.

The errors you encountered

The tools failed before the product screen could run; your expense form was not the cause.

IN USE TODAY

1. AGP 9.2.1 was newer than the old Studio supported

The previous Panda 3 IDE supported AGP only through 9.1.1. The React Native toolchain selected 9.2.1. Studio was upgraded to 2026.2.2; the project did not force AGP down to a different version.

2. 'Filename longer than 260 characters'

CMake 3.22.1 supplied an older Ninja that could not handle the long generated C++ object path. Windows long-path support was already enabled. A Windows-only config plugin now selects installed CMake 4.1.2 / Ninja 1.12.1 for both Studio and command-line builds.

3. 'Development servers' or 'Loading from 127.0.0.1'

The installed development launcher still needs Metro and a reachable connection. USB reverse was configured. Slow/stalled bundle loading recovered after restarting Metro with a cleared cache and reopening the client.

What was actually verified

ARM64 debug assembly passed in 9m24s, including the previously failing CMake task. The APK contains the ARM64 application library. The phone subsequently displayed onboarding, Home and Add Expense. This is not a release-performance or full physical-device QA result.

Navigation and application state

Navigation chooses the screen; state supplies its changing values.

IN USE TODAY

Technology	What it owns in today's app
Expo Router	Maps files under app/ to routes. home.tsx, transaction.tsx and other route files compose feature screens.
react-native-screens	Native screen primitives used by the navigation stack.
Safe Area Context	Insets that keep content away from system bars, cutouts and navigation areas.
React local state	An unfinished form amount, a visible error or an expanded filter panel.
React Context	Shared theme and system motion preference context.
Zustand	The small, shared in-memory preview store: profile, transactions, categories, budgets and goals.

One store, several views

Home, History and Insights read the same records. Saving in the form changes the store; subscribed screens recalculate their displays. The preview does not maintain a different financial database for each screen.

Today's route guards require a development build and preview profile to enter product screens. They prevent accidentally showing sample flows in a production build. They are not real authentication or server authorization.

Follow one expense

Imagine you manually record a food expense of ₹125.50.

IN USE TODAY

1. You enter text

The amount field contains the text 125.50; the category is Food.

2. Shared rules validate it

The code checks a positive amount, valid category/date and supported payment method.

3. Convert to integer paise

₹125.50 becomes 12,550 paise. Money calculations use bounded whole-number paise rather than accumulating decimal floating-point errors.

4. Update the in-memory store

A new transaction with an ID and timestamps enters the preview records. Duplicate presses are guarded.

5. Recalculate the views

The sample net spending ₹17,550 becomes ₹17,675.50. A ₹30,000 budget leaves ₹12,324.50. History gets a row and Insights changes.

6. Know the current limit

This update is in RAM only. It is not a bank payment, SQLite write or server-synced transaction.

The design technology

The visual language is shared code, not a collection of screenshots.

IN USE TODAY

Design tokens + StyleSheet

Tokens are named values for colour roles, typography, spacing, rounded corners and motion. React Native StyleSheet turns these choices into native component styling. NativeWind was an option in the brief, but it is not installed or required by today's implementation.

Manrope and expo-font

Manrope is the bundled font family. expo-font loads the font weights; the splash screen waits for font setup, with a fallback if loading fails. Consistent typography makes labels, amounts and helper text feel related.

SVG, icons and illustrations

SVG (Scalable Vector Graphics) describes shapes mathematically rather than as a fixed pixel grid. react-native-svg draws category icons, charts and original static Nim artwork. PNG files are used for app icons and screenshots.

Theme and accessibility roles

Roles such as ink, muted, primary and warning select purposeful colours in each theme. Text scaling, touch-target size, focus labels and reduced motion belong to usability, not just decoration.

The app uses actual interactive components. The pictures in this handbook document those components; screenshots are not the implementation of the app itself.

Motion and the mascot

Several technologies help movement, but each solves a different problem.

IN USE TODAY

Name	Its role and current status
Reanimated	In use for interface feedback, progress and transitions. Animations respect the reduced-motion policy.
Worklets	Small functions that can execute in a worklet runtime; used by the installed animation stack for responsive UI work.
Gesture Handler	Native gesture infrastructure; supports interactions more robustly than implementing every touch pattern manually.
Rive	Installed native runtime and fallback adapter. Intended for interactive mascot state-machine animation; original .riv asset rendering/bindings are not yet verified.
Nitro Modules	Native-module framework used by the selected Rive runtime. It connects JavaScript-facing objects with native implementations.
Hermes	Executes the app's JavaScript. A runtime, not a mascot renderer.
Lottie	An optional animation format/tool from the brief. Not installed or used in today's app.

Static Nim is the honest fallback

Saving, goal completion and budget attention have defined emotional meanings, but a successful native library build does not mean original character animation files exist. Common actions do not wait for a celebration animation to finish.

How we check the work

Different checks find different classes of mistakes.

IN USE TODAY

Tool / check	What it checks
TypeScript 6.0.3	Types, imports and consistency before execution.
ESLint + Hooks rules	Code mistakes, React-hook usage and project conventions.
typescript-eslint	TypeScript-aware linting configuration and rules.
Prettier 3.9.10	Consistent automatic code formatting.
Jest + jest-expo	Repeatable logic tests with Expo-compatible test configuration.
Expo Doctor / install check	Project health and dependency compatibility with the selected SDK.
ADB + UIAutomator	Native-device control, visible text assertions, screenshots and logs.
Contrast verifier	The project's selected text/background contrast pairs and token invariants.

Evidence, not a production-readiness badge

The current logic suite has 37 passing tests across four suites. Types, lint and formatting pass; the ARM64 build passed. Phase 3 recorded 52 design contrast checks. Some combined native test runs failed; focused reruns and limitations are documented rather than hidden.

Still outstanding: full TalkBack speech/focus review, authored Rive animation acceptance, release-device performance and the later security phase. One emulator input timeout under concurrent export was recorded; its root cause was not established.

Git, GitHub and configuration

Tools for maintaining the project are not the same as the running app.

IN USE TODAY

Git and GitHub

Git records meaningful local revisions called commits. GitHub hosts the remote repository. 'Commit' means record locally; 'push' means send commits to the remote. Having a GitHub repository does not automatically host an API or publish an Android app.

Configuration and secrets

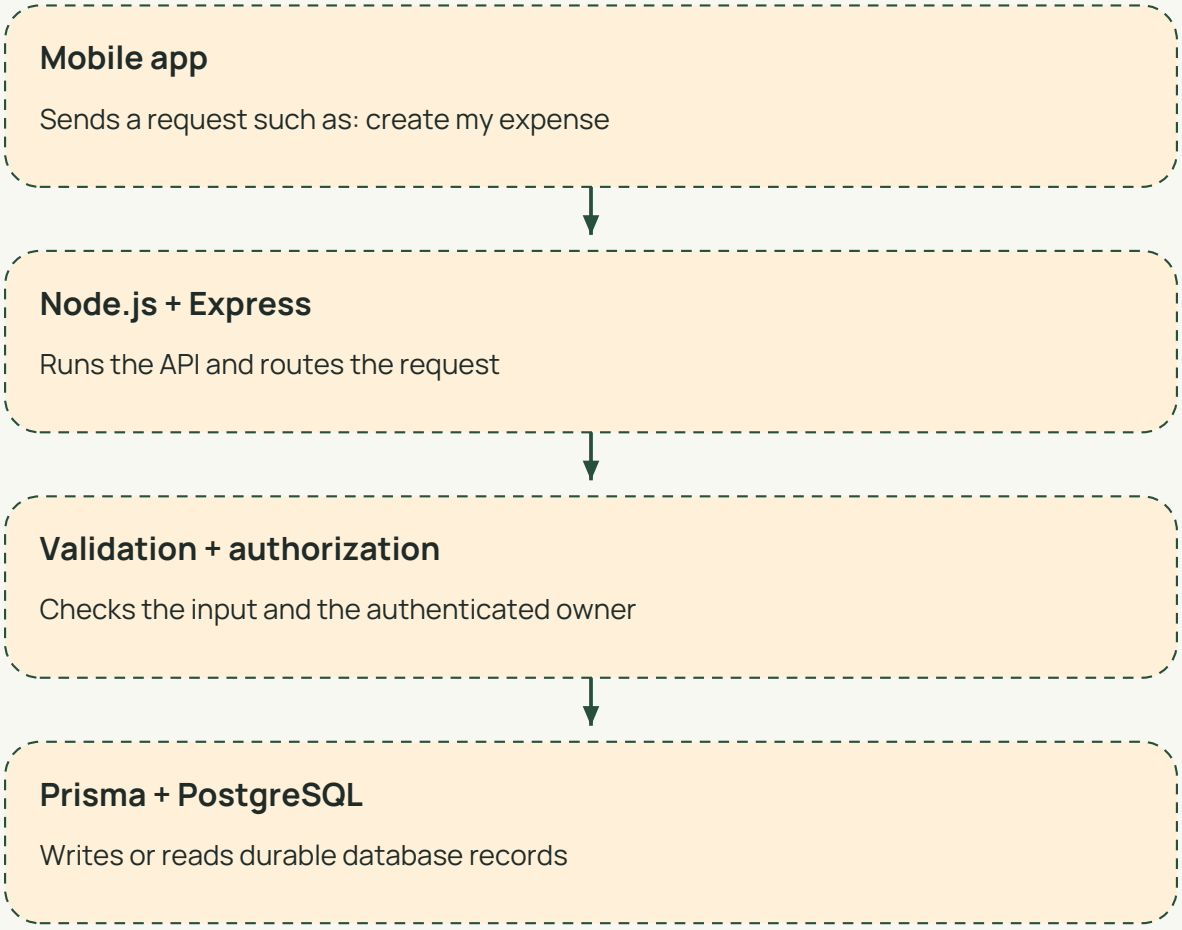
Environment settings describe which build/environment to use. Public mobile configuration can be inspected and cannot hide secrets. .gitignore keeps generated output and local machine settings out of version control; it is not encryption.

File / tool	Meaning in this project
.ts / .tsx	Typed logic / typed components with UI markup.
.cjs / .json	CommonJS tooling plugin / structured configuration or data.
.kt / .cpp / .so	Kotlin source / C++ source / compiled native library.
.riv / .apk	Rive asset file / installable Android package.
.env / local.properties	Environment overrides / local Android SDK path; handle local settings carefully.
CLAUDE.md / AI agent	Contributor instructions / an assistant that can edit and test code. Codex and Claude Code are development assistants, not app runtime libraries.

The future backend

A backend is a program that receives requests and enforces business rules.

PLANNED - NOT IMPLEMENTED



REST (Representational State Transfer) organizes resource-based HTTP APIs. GET reads; POST creates; PATCH edits; DELETE removes. **JSON** (JavaScript Object Notation) carries structured data. HTTPS protects it in transit.

SQL (Structured Query Language) queries relational databases. **PostgreSQL** is the proposed server database. **Prisma ORM** provides typed access and schema migrations. **Zod** validates input at runtime. Tables will link users and financial records.

The server must derive the user from authentication and scope every financial lookup to that owner. Database constraints and transactions protect consistency. Proposed endpoint/schema documents are designs, not deployed APIs.

The future real login

Today's form preview will later be replaced by genuine authentication.

PLANNED - NOT IMPLEMENTED

Password hashing

The proposed backend uses Argon2id to store a password hash rather than the original password. Hashing is a one-way check; it is not the same as encrypting and later reading a password.

JSON Web Token (JWT): access token

A short-lived signed credential lets the API identify an authenticated session. A signed JWT is not automatically encrypted, so secret financial data should not be placed in its readable payload.

Refresh token + SecureStore

A refresh credential obtains a new access token. The design rotates refresh tokens and detects reuse. Expo SecureStore is planned for protected credential storage on the device; it is not the financial database and is not integrated today.

Authorization, reset and logout

Login identifies the user; authorization decides which records that user may access. Real password reset needs an email-delivery flow. Logout must invalidate the right session and handle device data safely. Email-verification friction will be assessed per sensitive action.

No real authentication, token storage, password hashing service or email sending is implemented in the current preview. Attractive login screens alone do not provide security.

Saving offline and synchronizing

Temporary memory, local storage and a remote database serve different purposes.

PLANNED - NOT IMPLEMENTED

Layer	Meaning
Today's Zustand preview	RAM-only sample records. Not durable, not synchronized.
SQLite on the phone	A future local database that can retain entries after app restart and support offline transactions.
TanStack Query	Future API request/cache/retry state. It does not by itself become an offline ledger or a synchronization engine.
PostgreSQL on the server	Future durable cross-device/account data, accessed through the authorized API.

Offline-first save

Phase 6 will save the entry and a pending operation in SQLite atomically. The screen can confirm 'saved on this device' even while the network is unavailable. That is different from 'synced to the server'.

Outbox, retry and conflicts

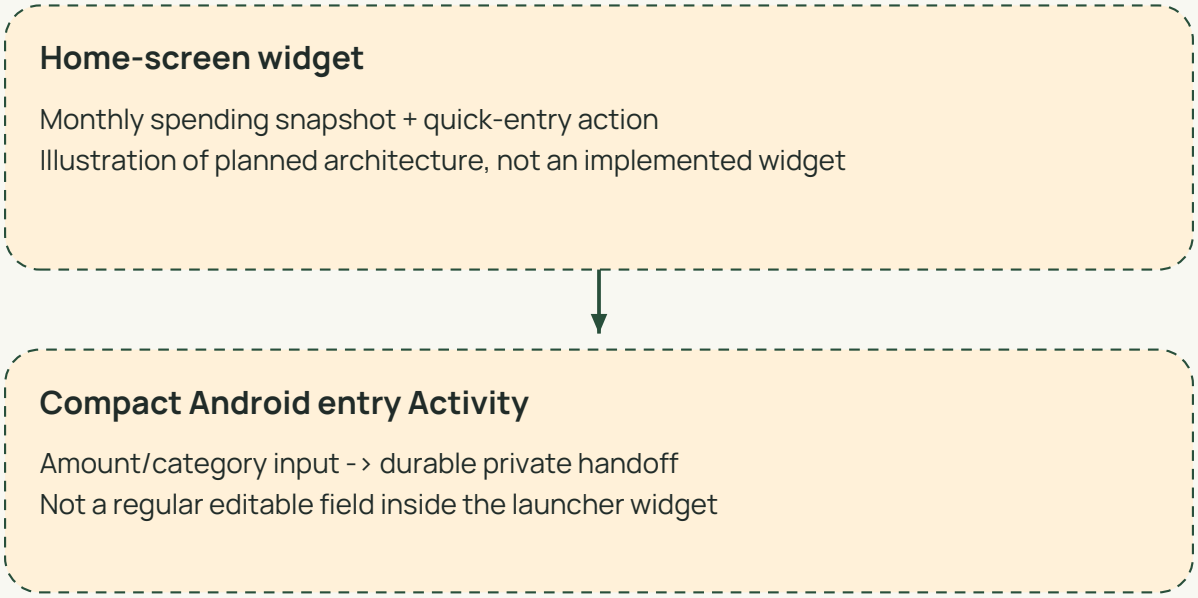
An outbox is a durable queue of pending changes. Stable operation IDs help avoid duplicates when retrying. Versions/conflict rules handle competing edits rather than silently overwriting money records. Background execution remains subject to Android's scheduling limits.

A basic personal tracker can work with local storage alone; a backend adds shared account services and cross-device records. In this project neither SQLite nor server sync is implemented yet. Keep these abstractions small until the approved phase needs them.

Widgets and future capture

A launcher widget is a small native surface with platform limits.

PLANNED - NOT IMPLEMENTED



Kotlin is the planned language for this isolated Android functionality. **Jetpack Glance** helps describe widget UI, but widgets have stricter interaction/update rules than normal app screens.

The design uses a small entry Activity for typing and a durable handoff to the application. It must handle the JavaScript process being stopped, account changes, retries and stale summaries. Locally saved and server-synced states must be distinct.

Future capture integrations are separate

UPI notification capture, bank SMS parsing, Account Aggregator and automatic categorization remain future possibilities. They are outside the initial release scope and are not silently enabled permissions or services in today's app.

Widget acceptance later requires both emulator and real-device testing. A drawing of a widget is not proof that the native feature works.

What is built, and what comes next

One phase at a time, with an explicit review before the next major phase.

REFERENCE - STATUS SNAPSHOT

Phase	Status / purpose
0 - Architecture	Proposal and technical foundation approved.
1 - Brand/design	Original Nivvi/Nim direction, tokens and design foundation approved; original Rive files remain missing.
2 - Mobile foundation	Approved native Expo foundation; documented asset/accessibility/device qualifications remain.
3 - Interactive UI	Implemented with mock data and scoped verification; awaiting review. Toolchain fixes support running it locally.
4 - Backend	Planned: PostgreSQL, Prisma, Express, real auth and owned REST APIs.
5 - Integration	Planned: connect the mobile UI to actual API/database records.
6 - Offline	Planned: SQLite durability, outbox, retries and synchronization.
7 - Widget	Planned: isolated Kotlin/Glance quick-entry feature.
8 - Polish	Planned: refined motion and real mid-range-device measurements.
9 - QA/security	Planned: broader isolation, security, accessibility and performance review.
10 - Release	Planned: production setup, signed Android build and approved deployment/publishing.

Not selected for today's implementation: NativeWind and Lottie. **Not implemented:** backend services, TanStack Query integration, SQLite, SecureStore authentication, native widget and banking automation. Web/iOS support is not a tested released product.

Installed mobile libraries - 1

Resolved installed versions, read from this workspace - not a claim about the latest releases.

REFERENCE - DIRECT DEPENDENCIES

Package	Installed	Why it is here
react	19.3.0	Component model and state-driven rendering.
react-native	0.88.1	Native mobile UI and platform integration.
expo	58.0.23	SDK and native/tooling integration.
expo-router	58.0.30	File-based screens and navigation.
expo-dev-client	58.0.18	Custom development launcher and client.
@expo/metro-runtime	58.0.17	Runtime support for Metro-based Expo apps.
zustand	5.1.0	Shared in-memory preview store.
react-native-reanimated	4.7.1	Native interface animation support.
react-native-worklets	0.13.0	Worklet runtime infrastructure.
react-native-gesture-handler	3.3.0	Native gesture handling.
@rive-app/react-native	0.5.12	Rive native runtime; original asset pending.
react-native-nitro-modules	0.36.5	Native module framework required by Rive.

These are the mobile project's direct runtime dependencies. Dependencies also bring smaller transitive packages, which are recorded in package-lock.json; they are not all separate product features.

Installed mobile libraries - 2

Resolved installed versions, read from this workspace - not a claim about the latest releases.

REFERENCE - DIRECT DEPENDENCIES

Package	Installed	Why it is here
react-native-svg	15.16.0	SVG icons, illustrations and charts.
react-native-safe-area-context	5.11.1	Screen safe-area inset handling.
react-native-screens	4.30.1	Native screen/navigation primitives.
@expo-google-fonts/manrope	0.4.3	Bundled Manrope font files.
expo-font	58.0.11	Font loading.
expo-splash-screen	58.0.13	Native splash-screen handoff.
expo-status-bar	58.0.4	Status-bar appearance.
expo-system-ui	58.0.7	System-level UI/background integration.
expo-linking	58.0.16	App URLs and deep-link helpers.
expo-constants	58.0.15	App/runtime configuration information.
react-dom	19.3.0	React renderer for the web target, not Android UI.
react-native-web	0.21.4	Web-compatible React Native components; not a released web product.

These are the mobile project's direct runtime dependencies. Dependencies also bring smaller transitive packages, which are recorded in package-lock.json; they are not all separate product features.

Developer-only helpers also include **tsx** to load TypeScript configuration, **@types/react**, **@types/node** and **@types/jest** for type definitions. Types describe APIs during development; they do not add a server or save records.

Your everyday run guide

For the current development build, the laptop and phone work together.

REFERENCE - PRACTICAL CHECKLIST

1. Start Metro on the laptop

```
cd D:\Projects\expense-tracker\apps\mobile
npx expo start --dev-client --localhost --port 8081 --max-workers 2
```

2. Connect the phone

Connect USB and approve the normal USB-debugging prompt if shown. In a terminal where adb is available, list devices and reverse the port. Replace YOUR_DEVICE_SERIAL with your phone's ID; it is needed when more than one device is connected.

```
adb devices
adb -s YOUR_DEVICE_SERIAL reverse tcp:8081 tcp:8081
```

3. Open Nivvi Dev

Android Studio can install/open the debug app. If you see Development servers, select the running Metro server or enter **http://127.0.0.1:8081** in the launcher's URL field. Wait for bundling to finish; leave Metro running.

4. Enter the product preview

Skip/finish the introduction. Use sample email **preview@example.com** and sample password **Preview123**, then Enter preview. Home exposes Add Expense/Add Income and the History, Insights and Goals tabs.

If loading stalls

Restart Metro with the same command plus --clear, wait until it says it is ready, then reopen the client. Reconnect USB reverse if needed. Do not run multiple Metro servers on port 8081. Switching to a release build is not a fix: today's production guard intentionally blocks the sample product routes.

Glossary: A to L

A word is easier to remember when you can connect it to one concrete job.

REFERENCE

ABI - the machine-code interface for a processor family, such as arm64-v8a.

ADB - the computer-to-Android debugging and communication tool.

AGP - Android Gradle Plugin; adds Android build tasks to Gradle.

API - Application Programming Interface: a way to request capabilities or data. A REST API is a network-facing kind.

APK - an installable Android application package.

Backend - a service that handles requests, accounts and authorized data operations.

Build / compile - turn source/configuration into executable output for a target.

Cache - a reusable copy kept to avoid doing the same work again; not automatically durable financial storage.

Component - a reusable piece of interface or behaviour.

Database - structured storage that can retain and query records.

Dependency - another package the project relies on.

Development client - our custom debug app that can connect to Metro.

Environment - the settings for a particular way of running/building the app.

Hermes - the engine that executes JavaScript in this mobile app.

Localhost - the device making the request itself; usually 127.0.0.1.

Glossary: M to Z

Keep the ownership of code, state and stored data separate.

REFERENCE

Metro - the development bundler/server supplying application code.

Migration - a versioned change to a database schema.

Monorepo - one repository containing related projects/packages.

Native - code or UI using the mobile platform's capabilities directly.

Node.js - a JavaScript runtime on the computer or a future backend server.

Offline-first - designing writes to succeed durably on-device before network synchronization.

ORM - Object-Relational Mapping: tooling that maps application structures to database operations.

Outbox - a durable queue of changes waiting to synchronize.

Port - a numbered network-service endpoint; Metro uses 8081 here.

RAM - temporary working memory; today's preview records live here.

Repository - a code project under Git, or in architecture a data-access boundary; context matters.

SDK / NDK - Software Development Kit / Native Development Kit; Android toolkits for application and C/C++ work.

State - values the interface reads and changes while it runs.

Token - a named design value, or in authentication a credential; these are different meanings.

Validation - checking that input follows rules before accepting it.

Widget - a limited Android launcher surface, separate from a full app screen.

Sources and project evidence

Project facts were checked locally; technology explanations use primary documentation.

REFERENCE

Snapshot evidence: root/mobile package manifests and installed package versions; app routes/config/plugins; preview store, finance rules and fixtures; Phase 3 review; architecture/database/API proposals; session handoff; ARM64 build record. Package versions and status are specific to this project snapshot.

Picture provenance: docs/review/phase3/*.png, captured from the actual Android development app on Pixel_7 API 34, 1080 x 2400. Illustrations of the development connection, backend and widget are explanatory vector diagrams created for this guide. Future diagrams are labelled planned.

Official reading links

01. React Native components
02. TypeScript handbook
03. Expo development builds
04. Expo Router
05. Node.js introduction
06. npm introduction
07. Android ADB
08. Studio / AGP compatibility
09. Windows CMake / Ninja paths
10. Hermes
11. Zustand
12. Reanimated glossary
13. Rive React Native runtime
14. Nitro Modules
15. Express
16. Prisma ORM concepts
17. PostgreSQL concepts
18. Zod validation
19. Expo SQLite
20. TanStack Query
21. Expo SecureStore
22. Jetpack Glance

Links were checked during preparation. Library documentation can evolve independently of this selected stack; future backend/widget versions must be checked when their phase is approved.

This PDF was authored with Python/ReportLab and rendered for visual review. Those are document-authoring tools, not Nivvi app dependencies. The project uses the bundled Manrope font under its existing SIL Open Font License.